

Analyse de données sur Python

Pr A. OUSAADANE

UMI, ENS

05/06/2026



Plan de cours

1 Introduction

2 Series

3 Dataframes

4 Étude de cas

1 Introduction

2 Series

3 Dataframes

4 Étude de cas

Introduction

- L'objectif de ce chapitre est d'introduire Python pour l'analyse de données, lorsqu'elles sont trop volumineuses pour la mémoire (RAM) d'un ordinateur.
- Cette étape est abordée par l'initiation aux fonctionnalités de la librairie **pandas** et à la classe **DataFrame**; lire et écrire des fichiers, gérer une table de données et les types des variables, échantillonner, discrétiser, regrouper des modalités, concaténation et jointure de tables...
- **Importance de l'analyse de données** : Utilisation dans les sciences, l'industrie, et la prise de décision.

Module Pandas

- Le module **pandas** a été conçu pour l'analyse de données. Il est particulièrement puissant pour manipuler des données structurées sous forme de tableau.
- Le module pandas n'est pas fourni avec la distribution Python de base. Il faut alors l'installer avant de l'importer.
- Pour charger pandas dans la mémoire de Python, on utilise la commande import habituelle :

```
import pandas
```

Par convention, on utilise pd comme nom raccourci pour pandas :

```
import pandas as pd
```

Structures de données

- **Series** : Une colonne de données avec un index.
- **DataFrame** : Un tableau avec plusieurs colonnes et index.
- **Chargement des données** : CSV, Excel, bases de données, etc.

1 Introduction

2 Series

3 Dataframes

4 Étude de cas

Series

Le premier type de données apporté par pandas est la **Series**, qui correspond à un vecteur à une dimension.

```
s = pd.Series([10, 20, 30, 40], index = ['a', 'b', 'c', 'd'])  
s  
  
a 10  
b 20  
c 30  
d 40  
dtype: int64
```


Sélection par étiquette ou indice

- Avec pandas, chaque élément de la série de données possède une étiquette qui permet d'appeler les éléments qui la composent.

```
s["a"]
```

```
10
```

- Pour accéder au premier élément par son indice (ici 0), comme on le ferait avec une liste, on utilise la méthode `.iloc` :

```
s.iloc[0]
```

```
10
```

- On peut extraire plusieurs éléments, par leurs indices ou leurs étiquettes :

```
s[["b", "d"]] #s.iloc[[1, 3]]
```

```
b 20
```

```
d 40
```

```
dtype: int64
```

Modifications de Series

Les étiquettes permettent de modifier et d'ajouter des éléments :

```
s["c"] = 300
```

```
s["z"] = 50
```

```
s
```

```
a 10
```

```
b 20
```

```
c 300
```

```
d 40
```

```
z 50
```

```
dtype: int64
```

Filtres

On peut filtrer une partie de la Series :

```
s[s>30]  
c 300  
d 40  
z 50  
dtype: int64
```

Remarque

- Cette écriture est pareille à celle des masques booléens dans le module NumPy.
- On peut aussi combiner plusieurs critères de sélection avec les opérateurs logiques & (pour ET) et | (pour OU).

- 1 Introduction
- 2 Series
- 3 Dataframes**
- 4 Étude de cas

Dataframes

- Dataframes correspondent à des tableaux à deux dimensions avec des étiquettes pour nommer les lignes et les colonnes.
- Création d'un Dataframe avec pandas à partir de données fournies comme liste de lignes :

```
import numpy as np
df = pd.DataFrame(columns=["a", "b", "c", "d"],
                   index=["Prof", "Médecin", "Banquier"],
                   data=[np.arange(10, 14),
                        np.arange(20, 24),
                        np.arange(30, 34)])
df
```

	a	b	c	d
Prof	10	11	12	13
Médecin	20	21	22	23
Banquier	30	31	32	33

Commentaires sur le code

- **Ligne 1.** On charge le module NumPy utilisé ensuite.
- **Ligne 2.** Le Dataframe est créé avec la fonction `DataFrame()` à laquelle on fournit plusieurs arguments. L'argument `columns` indique le nom des colonnes, sous forme d'une liste.
- **Ligne 3.** L'argument `index` définit le nom des lignes, sous forme de liste également.
- **Lignes 4 à 6.** L'argument `data` fournit le contenu du Dataframe, sous la forme d'une liste de valeurs correspondantes à des lignes. Ainsi, `np.arange(10, 14)` qui est équivalent à `[10, 11, 12, 13]` correspond à la première ligne du Dataframe.

Le même Dataframe peut aussi être créé à partir des valeurs fournies en colonnes sous la forme d'un dictionnaire :

```
data = {"a": np.arange(10, 40, 10),  
        "b": np.arange(11, 40, 10),  
        "c": np.arange(12, 40, 10),  
        "d": np.arange(13, 40, 10)}  
df = pd.DataFrame(data)  
df.index = ["Prof", "Médecin", "Banquier"]  
df
```

	a	b	c	d
Prof	10	11	12	13
Médecin	20	21	22	23
Banquier	30	31	32	33

- **Lignes 1 à 4.** Le dictionnaire data contient les données en colonnes. La clé associée à chaque colonne est le nom de la colonne.
- **Ligne 5.** Le dataframe est créé avec la fonction `pd.DataFrame()` à laquelle on passe data en argument.
- **Ligne 6.** On peut définir les étiquettes des lignes de n'importe quel dataframe avec l'attribut `df.index`.

Quelques propriétés

- Les dimensions d'un dataframe sont données par l'attribut `.shape` :

```
df.shape
```

```
(3, 4)
```

Le dataframe `df` possède trois lignes et quatre colonnes.

- L'attribut `.columns` renvoie le nom des colonnes et permet aussi de renommer les colonnes d'un dataframe :

```
df.columns
```

```
Index(['a', 'b', 'c', 'd'], dtype='object')
```

```
df.columns=["Rabat", "Meknès", "Marrakech", "Tanger"]
```

```
df
```

	Rabat	Meknès	Marrakech	Tanger
Prof	10	11	12	13
Médecin	20	21	22	23
Banquier	30	31	32	33

- La méthode `.head(n)` renvoie les n premières lignes du Dataframe (par défaut, n vaut 5) :

`df.head(2)`

	Rabat	Meknès	Marrakech	Tanger
Prof	10	11	12	13
Médecin	20	21	22	23

- **Sélection de colonnes** : On peut sélectionner une colonne par son étiquette :

```
df["Meknès"]
```

```
Prof 11
```

```
Médecin 21
```

```
Banquier 31
```

- La notation `df["Meknès"]` sélectionne une colonne et renvoie un objet `Series` :

```
type(df["Meknès"])
```

```
#pandas.core.series.Series
```

- Pour sélectionner plusieurs colonnes, il faut fournir une liste de noms de colonnes :

```
df[["Meknès", "Marrakech"]]
```

	Meknès	Marrakech
Prof	11	12
Médecin	21	22
Banquier	31	32

- On obtient cette fois un Dataframe avec les colonnes sélectionnées :

```
type(df[["Meknès", "Marrakech"]])  
#pandas.core.frame.DataFrame
```

Remarque

La sélection de plusieurs colonnes nécessite une liste entre les crochets. Si on utilise un tuple du type `df[("Meknès", "Marrakech")]`, Python renvoie une erreur `KeyError: ('Meknès', 'Marrakech')`.

- **Sélection de lignes** : Pour sélectionner une ligne, il faut utiliser l'instruction `.loc` et l'étiquette de la ligne :

```
df.loc["Prof"]
```

```
Rabat      10
Meknès     11
Marrakech  12
Tanger     13
Name: Prof, dtype: int64
```

- On peut aussi sélectionner plusieurs lignes :

```
df.loc[["Prof", "Banquier"]]
```

	Rabat	Meknès	Marrakech	Tanger
Prof	10	11	12	13
Banquier	30	31	32	33

- On peut aussi sélectionner des lignes avec l'instruction `.iloc` et l'indice de la ligne (la première ligne ayant l'indice 0) :

```
df.iloc[1]
```

```
Rabat      20
Meknès     21
Marrakech  22
Tanger     23
Name: Médecin, dtype: int64
```

```
df.iloc[[1,0]]
```

	Rabat	Meknès	Marrakech	Tanger
Médecin	20	21	22	23
Prof	10	11	12	13

- On peut également utiliser les tranches (comme pour les listes) :

```
df.iloc[0:2]
```

- **Sélection sur les lignes et les colonnes** : On peut bien sûr combiner les deux types de sélection (en ligne et en colonne) :

```
df.loc["Prof", "Tanger"]  
13
```

```
df.loc[["Prof", "Médecin"], ["Marrakech", "Tanger"]]
```

- **Sélection par condition** : Sélectionnons maintenant toutes les lignes pour lesquelles les effectifs à Tanger sont supérieurs à 15 :

```
df[ df["Tanger"]>15 ]
```

- De cette sélection, on ne souhaite garder que les valeurs pour Meknès :

```
df[ df["Tanger"]>15 ]["Meknès"]
```

- On peut aussi combiner plusieurs conditions avec & pour l'opérateur **et** et | pour l'opérateur **ou** :

```
df[ (df["Tanger"]>15) & (df["Marrakech"]>25) ]  
df[ (df["Tanger"]>15) | (df["Marrakech"]>25) ]
```

Combinaison de dataframes

- On a souvent besoin de combiner deux tableaux à partir d'une colonne commune. Par exemple, si on considère les deux dataframes suivants :

```
data1 = {"Meknès": [10, 23, 17], "Rabat": [3, 15, 20]}  
df1 = pd.DataFrame.from_dict(data1)  
df1.index = ["Prof", "Médecin", "Banquier"]  
df1
```

```
data2 = {"Tanger": [3, 9, 14], "Fès": [5, 10, 8]}  
df2 = pd.DataFrame.from_dict(data2)  
df2.index = ["Prof", "Banquier", "Agent de sécurité"]  
df2
```

- On souhaite combiner ces deux dataframes. On remarque déjà qu'il y a des Prof à Meknès et Rabat, mais pas d'agent de sécurité et qu'il y a des agents de sécurité à Tanger et Fès, mais pas de Médecin.

- Pandas propose pour cela la fonction `concat()`, qui prend comme argument une liste de dataframes :

`pd.concat([df1, df2])`

	Meknès	Rabat	Tanger	Fès
Prof	10.0	3.0	NaN	NaN
Médecin	23.0	15.0	NaN	NaN
Banquier	17.0	20.0	NaN	NaN
Prof	NaN	NaN	3.0	5.0
Banquier	NaN	NaN	9.0	10.0
Agent de sécurité	NaN	NaN	14.0	8.0

- NaN indique des valeurs manquantes, cela signifie littéralement Not a Number. Mais le résultat obtenu n'est pas celui que nous attendions, puisque les lignes de deux dataframes ont été recopiées.

- L'argument supplémentaire `axis=1` produit le résultat attendu :

```
pd.concat([df1, df2], axis=1)
```

	Meknès	Rabat	Tanger	Fès
Prof	10.0	3.0	3.0	5.0
Médecin	23.0	15.0	NaN	NaN
Banquier	17.0	20.0	9.0	10.0
Agent de sécurité	NaN	NaN	14.0	8.0

- Si on ne souhaite conserver que les lignes communes aux deux dataframes, il faut ajouter l'argument `join="inner"` :

```
pd.concat([df1, df2], axis=1, join="inner")
```

	Meknès	Rabat	Tanger	Fès
Prof	10	3	3	5
Banquier	17	20	9	10

- Créons un Dataframe composé de nombres aléatoires compris entre 100 et 200, répartis en trois colonnes (a, b et c) et 1000 lignes :

```
nb_rows = 1000
df = pd.DataFrame(
{
    "a": np.random.randint(100, 200, nb_rows),
    "b": np.random.randint(100, 200, nb_rows),
    "c": np.random.randint(100, 200, nb_rows),
}
)
```

- On peut vérifier que le dataframe a bien les propriétés attendues en utilisant les attributs `df.shape` et `df.head`.

- On souhaite créer une nouvelle colonne (d) qui sera le résultat de la multiplication des colonnes a et b, à laquelle on ajoute ensuite la colonne c.
- Une première manière de faire est de procéder ligne par ligne. La méthode `.iterrows()` permet de parcourir les lignes d'un Dataframe et renvoie un tuple contenant l'indice de la ligne (sous la forme d'un entier) et la ligne elle-même (sous la forme d'une Series) :

```
for idx, row in df.iterrows():  
    df.at[idx, "d"] = (row["a"] * row["b"]) + row["c"]
```

- Cette approche n'est pas optimale car elle est très lente, le code s'exécute en moyenne 52,4 ms.

- Une autre approche, plus efficace, consiste à réaliser les opérations directement sur les colonnes (et non plus ligne par ligne) :

```
df["d"] = (df["a"] * df["b"]) + df["c"]
```

Le code s'exécute en moyenne en $250\ \mu s$, soit environ 200 fois plus rapidement qu'avec `iterrows`.

Remarque

Tout comme avec les arrays de Numpy, les opérations vectorielles avec les Dataframes sont rapides et efficaces. Privilégiez toujours ce type d'approche avec les arrays de NumPy ou les Series et Dataframes de pandas.

Chargement d'un fichier CSV

- Une fonctionnalité très intéressante de pandas est d'ouvrir très facilement un fichier au format .csv :

```
# Chargement d'un fichier CSV
```

```
df = pd.read_csv('data.csv')
```

```
# Aperçu des premières lignes
```

```
print(df.head())
```

- Le contenu est chargé sous la forme d'un Dataframe dans la variable df.

- 1 Introduction
- 2 Series
- 3 Dataframes
- 4 Étude de cas**